1.0 Introduction 3
2.0 PART A : Case Study Analysis 3
2.1 Problem Statement 3
2.2 SMART objectives 4
2.3 Target Users 4
2.4 Assumptions and Constraints : 5
3. PART B : IPO Analysis and Algorithm 5
3.1 IPO Analysis 5
3.2 Algorithm 7
4. PART C Modular Design , Pseudocode and Flowchart 7
4.1 Pseudocode for Main Program 8
4.2 Pseudocode for Updating Daily Usage 9
4.3 Pseudocode for Electricity Calculation 10
PART D : Application Features and Testing 14
PART E:Consistentcy and Enhancement 15
PART F: REFLECTION 16

LIST OF FIGURES

Figure 1 ENGINEERING AND IT STUDENT 4
Figure 2 8
Figure 3 9
Figure 4 10
Figure 5 FC 1 11
Figure 6 FC 2 12
Figure 7 FC 3 13

1.0 Introduction

Technology is widely used in modern workshops to improve efficiency, safety, and cost management. In engineering workshops, different types of machines are used daily for practical activities and project work. However, without a proper monitoring system, it can be difficult to track machine usage, electricity consumption, and safety conditions such as overheating. Manual monitoring methods are often time-consuming and may lead to inaccurate records.

This project introduces the **Smart Workshop Energy & Safety Monitoring System (SWESMS)**. The system is developed using C++ programming language and operates through a console-based interface. It allows users to register workshop equipment, record daily usage, monitor temperature levels, and calculate electricity consumption.

The system also includes safety features that check whether a machine's temperature exceeds its maximum limit. If the temperature becomes too high, the system will generate warning or critical alerts. In addition, the system calculates the total electricity usage in kilowatt-hours (kWh) and estimates the daily electricity cost.

Overall, this system helps improve workshop management by organizing equipment records, monitoring machine safety, and estimating operational costs in a more systematic way.

2.0 PART A : Case Study Analysis

2.1 Problem Statement

Workshops used for engineering and IT practical sessions often rely on multiple machines such as drills, cutters, and other electrical equipment. These machines consume electricity and generate heat when operating for long periods of time. Without proper monitoring, it can be difficult to manage energy usage and ensure that machines are operating safely.

Currently, many workshops rely on manual observation or simple records to track machine usage. This approach has several problems. First, electricity consumption is not calculated automatically, making it difficult to estimate daily operating costs. Second, there is no structured method to monitor machine temperature levels. If a machine overheats, it may cause damage to the equipment or create safety risks for users. Third, maintenance scheduling may be ignored if total machine runtime is not properly tracked.

Because of these issues, a monitoring system is needed to manage equipment information, record daily usage, and provide safety alerts. The Smart Workshop Energy & Safety

Monitoring System (SWESMS) is designed to solve these problems by allowing users to register equipment, update daily usage data, check machine safety status, and generate energy cost reports automatically.

2.2 SMART objectives

OBJECTIVE 1
To develop a system that can register and store up to 30 workshop equipment records including equipment ID, type, power consumption, temperature limit, and service interval.

2.3 Target Users

The main users of the Smart Workshop Energy & Safety Monitoring System are workshop staff and students who use workshop equipment during practical sessions.

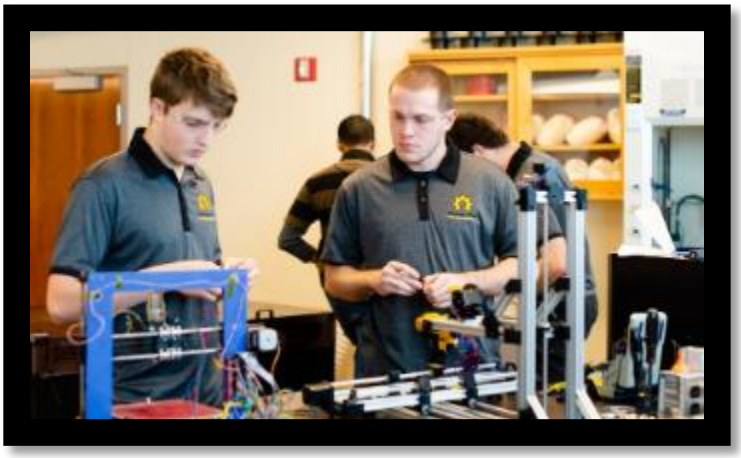

Figure 1 ENGINEERING AND IT STUDENT

Engineering or IT Students

Students who use the workshop machines can also benefit from the system because it provides clear information about machine status and safety alerts.

2.4 Assumptions and Constraints :

Assumptions	Constrains
The workshop contains a maximum of 30 machines that need to be recorded in the system.	The system is console-based and does not include a graphical user interface.
The electricity rate used in the system is fixed at RM0.50 per kilowatt-hour.	The system only stores data temporarily while the program is running and does not use a database for permanent storage.
Users will manually enter the machine usage hours and temperature readings each day.	The accuracy of the monitoring results depends on the correctness of the user's input data.

3. PART B : IPO Analysis and Algorithm

3.1 IPO Analysis

The Input–Process–Output (IPO) model helps explain how the system receives data, processes it, and produces meaningful results. In the Smart Workshop Energy & Safety Monitoring System (SWESMS), the IPO concept is applied to manage workshop equipment, monitor safety conditions, and calculate electricity consumption.

INPUT : **Equipment info**

Equipment ID , Equipment category or type,Power consumption in watts, Maximum temperature limit, Service interval in hourS

Daily Usage Data

Hours used in a day,Current temperature of the machin

PROCESS

1. The system stores equipment information inside an array structure.
2. It searches for a specific equipment ID when the user wants to update daily data.
3. It validates the daily usage hours to ensure the value is between 0 and 24.
4. The system checks the machine temperature against its maximum heat limit to determine its safety status.

5. The system calculates energy consumption using the formula:

$$Energy\ (kWh) = \left(Power\ in\ Watts — 1000\right) \times\ Hours\ Used$$

6. The total energy consumption of all machines is calculated.
7. If the total energy exceeds 8 kWh, a 20% surcharge is added to the final electricity cost.

OUTPUT

The system produces several outputs for the user.

- Confirmation messages when equipment is successfully registered
- Updated machine usage information
- Safety status of machines (NORMAL, WARNING, CRITICAL)
- Alerts such as OVERHEAT or maintenance due
- Total daily energy consumption
- Estimated electricity cost

3.2 Algorithm

→ Start the program.

→ Display the main system menu.

→ Ask the user to enter a menu option.

→ If the user selects option 1, the system will allow the user to register a new equipment record.

→ If the user selects option 2, the system will update the daily usage data for a specific machine.

→ If the user selects option 3, the system will check and display the safety status of all machines.

→ If the user selects option 4, the system will calculate the total electricity consumption and display the cost report.

→ If the user selects option 5, the program will terminate.

→ If the user enters an invalid option, an error message will be displayed.

→ The system will continue displaying the menu until the user chooses to exit.

→ End program.

4. PART C Modular Design , Pseudocode and Flowchart

The system is designed using a modular approach where different tasks are separated into individual functions. This approach helps make the program easier to understand, maintain, and debug.

Add New Equipment	Update Daily Usage	Check Safety Status	Calculate Electricity Cost

4.1 Pseudocode for Main Program

```
START

DECLARE machine array
DECLARE totalItems = 0

REPEAT
    DISPLAY system menu
    DISPLAY options 1–5
    GET user choice

    IF choice = 1 THEN
        CALL AddNewItem

    ELSE IF choice = 2 THEN
        CALL UpdateDailyData

    ELSE IF choice = 3 THEN
        CALL CheckStatus

    ELSE IF choice = 4 THEN
        CALL CalculateElectricity

    ELSE IF choice = 5 THEN
        DISPLAY "Closing system"

    ELSE
        DISPLAY "Invalid input"

UNTIL choice = 5

END
```

Figure 2

4.2 Pseudocode for Updating Daily Usage

```
FUNCTION UpdateDailyData

ASK user to enter equipment ID
SEARCH equipment list for matching ID

IF ID found THEN
    REPEAT
        ASK user for hours used today
        IF hours < 0 OR hours > 24
            DISPLAY error message
        ENDIF
    UNTIL hours between 0 and 24

    STORE hours in todayHours

    ASK user for current temperature
    STORE temperature

    ADD hours to total runtime

    DISPLAY update confirmation

ELSE
    DISPLAY "ID not found"

ENDIF

END FUNCTION
```

Figure 3

.

4.3 Pseudocode for Electricity Calculation

```
FUNCTION CalculateElectricity

SET totalEnergy = 0

FOR each machine in the list
    CALCULATE machineEnergy
    machineEnergy = (watts / 1000) x hours

    ADD machineEnergy to totalEnergy
END FOR

SET cost = totalEnergy x 0.50

IF totalEnergy > 8
    surcharge = cost x 0.20
    cost = cost + surcharge
ENDIF

DISPLAY total energy
DISPLAY final cost

END FUNCTION
```

Figure 4

4.1 System Flowchart

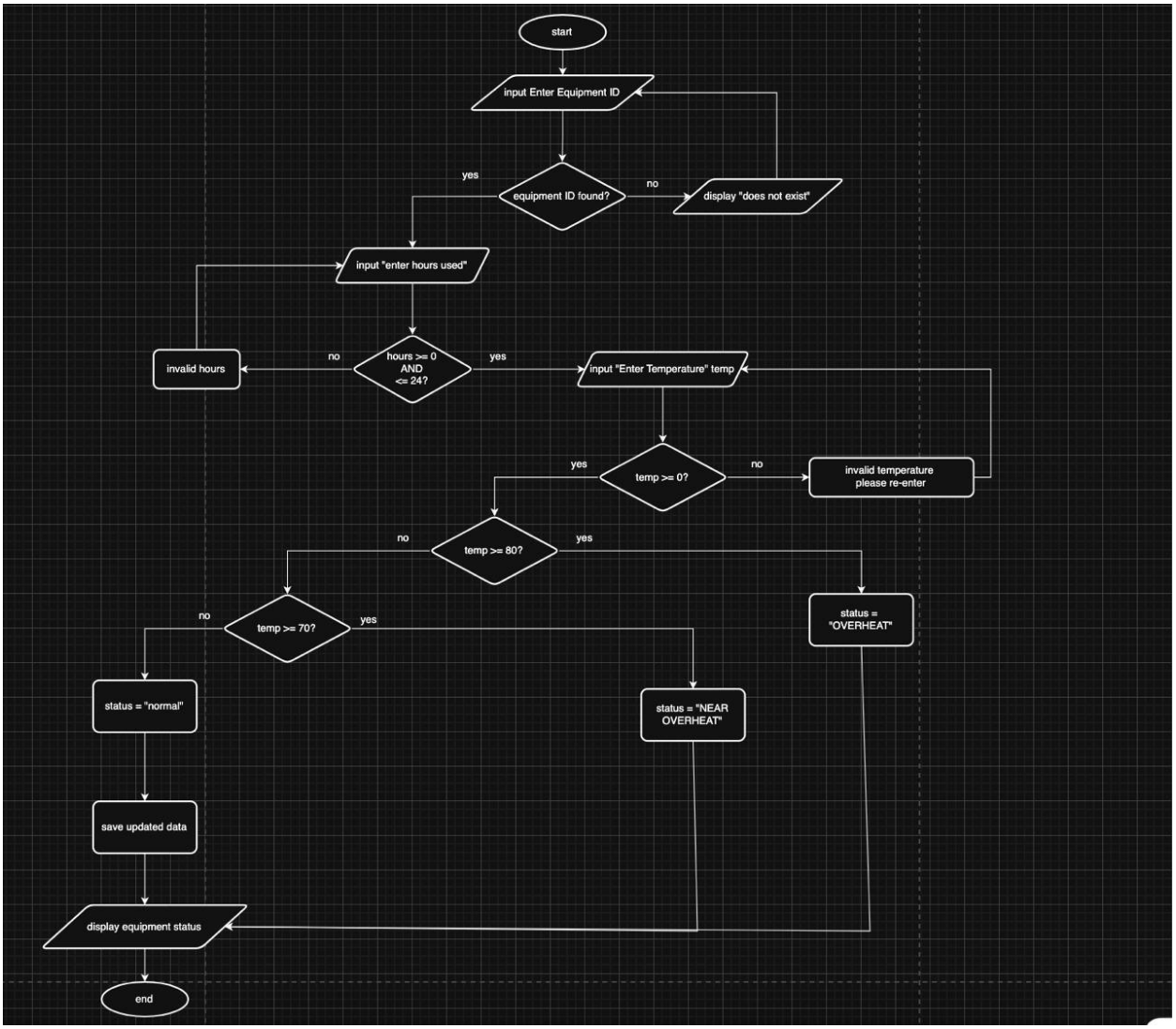

Figure 5 FC 1

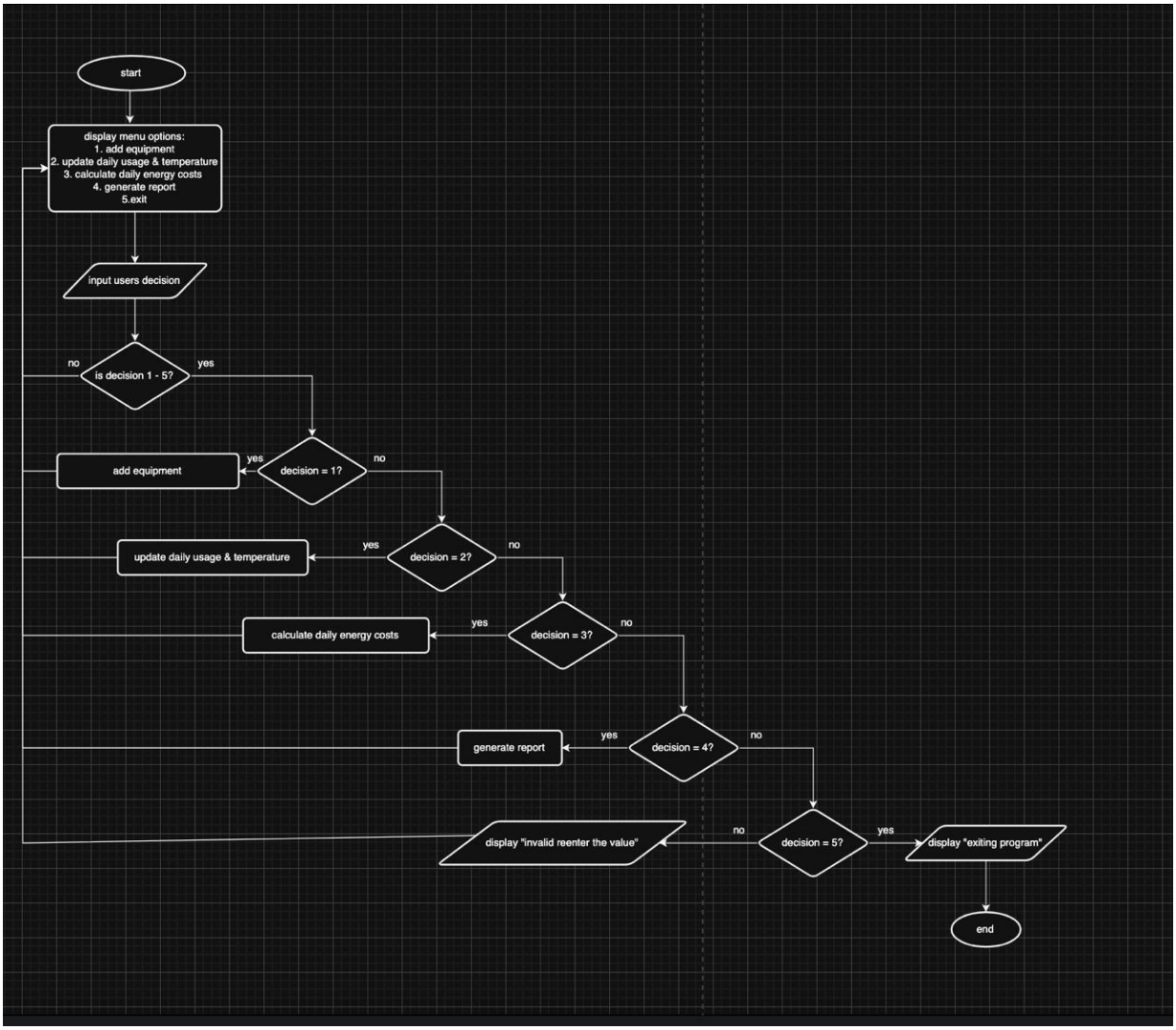
start
display menu options:
1. add equipment
2. update daily usage & temperature
3. calculate daily energy costs
4. generate report
5.exit
input users decision
is decision 1 - 5?
no
yes
decision = 1?
yes
no
add equipment
decision = 2?
yes
no
update daily usage & temperature
decision = 3?
yes
no
calculate daily energy costs
decision = 4?
yes
no
generate report
decision = 5?
no
yes
display "invalid reenter the value"
display "exiting program"
end

Figure 6 FC 2

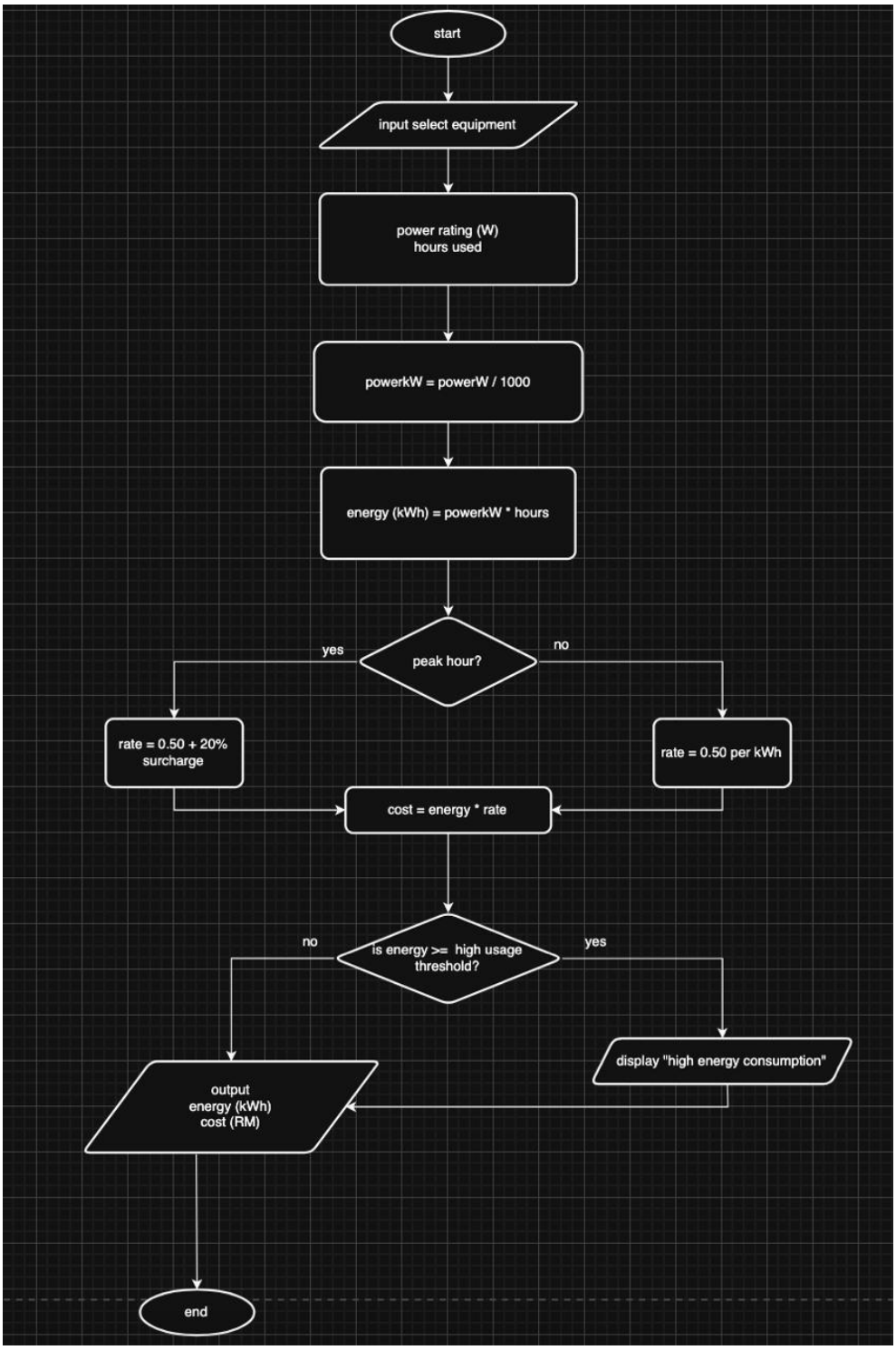
start
input select equipment
power rating (W)
hours used
powerkW = powerW / 1000
energy (kWh) = powerkW * hours
peak hour?
yes
no
rate = 0.50 + 20% surcharge
rate = 0.50 per kWh
cost = energy * rate
is energy >= high usage threshold?
no
yes
display "high energy consumption"
output
energy (kWh)
cost (RM)
end

Figure 7 FC 3

PART D : Application Features and Testing

SEGi College Subang Jaya

COURSE CODE- COURSE NAME

TEST CASE

PROJECT TITLE:			
Test Case ID:		3 expert user	can ask your lecturers to test your system
Tester Name:		3 normal user	can be anyone
Tester's Background:			
a) Area of specialization:			
b) Position/Job:			
c) Industry:			
d) Age:			

STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	Add Equipment	addNewItem()	User registers new equipment	ID: EQ04 Type: Grinder Power: 1000 MaxHeat: 85	System successfully adds new equipment	Equipment added successfully	PASS	
2	Update Usage	updateDailyData()	User updates today's machine usage	ID: EQ01 Hours: 4 T	System updates runtime and temperature	Data Updated	PASS	
3	Invalid Input	updateDailyData()	User enters invalid hours	Hours: 30	System shows error message	Error displayed	PASS	
4	Safety Status	checkStatus()	Machine temperature exceeds limit	Temp: 90 MaxHeat: 1	System shows CRITICAL alert	CRITICAL displayed	PASS	
5	Energy Report	calcElectricity()	System calculates energy usage	Power: 1200W Hour	Correct energy and cost shown	Correct result, --- DAILY ENERGY REPORT --- Total Energy: X kWh Final Total Cost: RM X	PASS	
6	Exit Programt	main()	User selects exit option	Input: 5	Program terminates	Program closed	PASS	

PART E:Consistentcy and Enhancement

Although the system works correctly , there are several improvements that could implemented in future One possible improvement is adding a graphical user interface (GUI). A GUI-based system would be easier for users to interact with compared to a console program. Buttons, menus, and visual displays could make the system more user-friendly.

Another enhancement would be integrating the system with real sensors. Instead of manually entering machine temperature and usage hours, sensors could automatically send data to the system. This would improve the accuracy of monitoring and reduce human error.

PART F: REFLECTION

This project shows how a simple monitoring system can be developed using basic programming concepts. Structures, arrays, loops, and functions were used to build the program. These concepts helped organize machine data and allowed the system to perform different tasks such as storing information, displaying data, and checking machine conditions.

One important lesson from this project is sharing ideas with engineering student (YX) during assignment discussions. Through discussion, different ideas and solutions were exchanged. This helped improve understanding of the system design and programming logic. It also showed that teamwork and communication are important when solving technical problems.

Overall, the project helped strengthen understanding of programming concepts and showed how these concepts can be applied to create a simple system for monitoring machines.